



Algorithmique et Programmation en C

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction à l'algorithmique et au langage C

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

Un algorithme est une suite **finie** et **non ambiguë** d'instructions permettant de résoudre un problème ou d'accomplir une tâche à partir de données d'entrée, en produisant un résultat en un temps fini. Cette définition, en apparence simple, structure toute la démarche de programmation : avant même d'écrire une seule ligne de code, il faut être capable de décrire précisément *quoi faire, dans quel ordre, et jusqu'à quand*. Un algorithme n'est donc pas propre à un langage de programmation particulier — il précède le code et peut, en théorie, être traduit dans n'importe quel langage.

Trois propriétés caractérisent formellement un algorithme :

- **La finitude** : l'exécution doit se terminer après un nombre fini d'étapes. Un traitement qui ne s'arrête jamais (par exemple une boucle mal conçue) n'est pas, au sens strict, un algorithme — c'est un défaut de conception qui, en sécurité, peut se traduire par un déni de service.
- **Le déterminisme** : chaque étape doit être définie de façon univoque, sans ambiguïté d'interprétation. Deux exécutions avec les mêmes données d'entrée doivent produire exactement le même résultat (sauf si l'algorithme intègre volontairement un élément aléatoire, auquel cas cela doit être explicite).
- **L'effectivité** : chaque étape doit être suffisamment simple et concrète pour être réellement exécutable, que ce soit par un humain avec un crayon et du papier ou par une machine. « Trouver le mot de passe optimal » n'est pas une étape effective ; « comparer deux caractères » l'est.

Avant d'écrire du code, il est vivement recommandé de décrire un algorithme en **pseudo-code** : une notation semi-formelle, indépendante de tout langage de programmation, qui permet de raisonner sur la logique du problème sans se soucier des détails de syntaxe. Cette étape de conception, souvent négligée par les débutants pressés d'ouvrir leur éditeur de code, évite une grande partie des erreurs de logique et facilite grandement le débogage ultérieur.

```
ALGORITHME Maximum
ENTRÉE : a, b (deux entiers)
SORTIE : le plus grand des deux
DÉBUT
    SI a > b ALORS
        RETOURNER a
    SINON
        RETOURNER b
    FIN SI
FIN
```

Ce pseudo-code décrit sans ambiguïté la logique à suivre pour comparer les deux valeurs et retourner la plus grande. Sa traduction en C, en Python ou en Java changera de syntaxe mais conservera exactement la même structure logique — c'est précisément l'intérêt de séparer la phase de

2. Pourquoi le langage C ?

Le langage C, créé par Dennis Ritchie chez Bell Labs en 1972 pour réécrire le système Unix, reste aujourd'hui l'un des langages les plus enseignés et les plus utilisés au monde, en particulier dans les domaines proches du matériel et de la sécurité. Plusieurs raisons expliquent ce choix pédagogique pour un cursus en sécurité informatique :

- **Accès direct à la mémoire** : le C offre un contrôle explicite sur l'allocation, l'organisation et la manipulation de la mémoire via les pointeurs (chapitre 6). Ce contrôle, absent ou fortement encadré dans des langages comme Python ou Java, est indispensable pour comprendre en profondeur des classes entières de vulnérabilités — débordement de tampon (*buffer overflow*), utilisation après libération (*use-after-free*), fuite mémoire — qui restent parmi les causes les plus fréquentes de failles critiques dans les logiciels réels.
- **Omniprésence dans les systèmes bas niveau** : une part considérable des systèmes d'exploitation (le noyau Linux, une grande partie de Windows), des pilotes matériels, des systèmes embarqués et des outils réseau est écrite en C. Comprendre le C est donc un prérequis pour analyser, auditer ou sécuriser ces composants critiques de l'infrastructure numérique.
- **Passerelle vers l'analyse bas niveau** : la maîtrise du C facilite considérablement la lecture de code assembleur, de code décompilé ou désassemblé — des compétences centrales en analyse de vulnérabilités, en rétro-ingénierie (*reverse engineering*) et en cryptanalyse appliquée, matières abordées plus loin dans le cursus.
- **Performance et prévisibilité** : le C ne masque pas ce qui se passe « sous le capot ». Le code compilé correspond de façon relativement directe à ce que fait le processeur, ce qui en fait un excellent langage pour apprendre à raisonner sur le coût réel (temps, mémoire) d'un programme — une compétence qui reste utile même lorsqu'on programme ensuite dans des langages de plus haut niveau.

Ce contrôle bas niveau a une contrepartie : le C ne protège presque jamais le programmeur contre ses propres erreurs. Contrairement à des langages managés qui vérifient automatiquement les débordements de tableau ou gèrent la mémoire à la place du développeur, le C considère que c'est au programmeur d'assurer la correction et la sécurité de son code. C'est précisément cette absence de filet de sécurité qui rend le C si pertinent pour un cours de sécurité informatique : comprendre où et pourquoi le langage laisse la porte ouverte à des erreurs, c'est comprendre l'origine d'une grande partie des vulnérabilités logicielles rencontrées dans le monde réel.

3. De l'algorithme au programme exécutable

Un fichier source C (.c) n'est pas directement exécutable : il doit passer par une chaîne de transformations, appelée **chaîne de compilation**, avant de devenir un programme que le système d'exploitation peut lancer. Comprendre ces étapes aide à situer où et comment certaines erreurs (de syntaxe, de type, ou de liaison) sont détectées.

1. **Préprocesseur** : avant toute analyse du langage C proprement dit, le préprocesseur traite les directives qui commencent par `#` — `#include` (incorporation textuelle du contenu d'un autre fichier, typiquement un en-tête .h), `#define` (définition de macros et de constantes symboliques), `#ifdef/#ifndef` (compilation conditionnelle, utile par exemple pour adapter le code à différentes plateformes). Le résultat de cette étape est un fichier source « étendu », purement textuel, sans encore aucune notion de type ni de syntaxe C validée.
2. **Compilation** : le compilateur analyse la syntaxe du code C, vérifie la cohérence des types, puis traduit le programme en code assembleur, propre au processeur cible, avant de le convertir en **code objet** (fichier .o) — un format binaire contenant des instructions machine mais pas encore un programme complet et autonome.
3. **Édition de liens (linking)** : l'éditeur de liens assemble un ou plusieurs fichiers objets ainsi que les bibliothèques nécessaires (par exemple la bibliothèque standard C, qui fournit `printf`, `malloc`, etc.) pour produire un unique fichier exécutable. C'est à cette étape que sont résolues les références à des fonctions définies dans d'autres fichiers ou bibliothèques ; une fonction déclarée mais jamais définie provoque une erreur de liaison (« *undefined reference* »).

```
gcc -Wall -Wextra -std=c11 -o programme programme.c
./programme
```

Dans cette commande, `gcc` est le compilateur GNU pour le C ; `-Wall -Wextra` active un ensemble étendu d'avertissements (variables inutilisées, comparaisons suspectes, types incompatibles...) qui ne sont pas des erreurs bloquantes mais signalent presque toujours un problème réel dans le code ; `-std=c11` fixe la version du standard C utilisée pour la compilation (ici C11) ; `-o programme` nomme le fichier exécutable produit. La seconde ligne, `./programme`, exécute le programme ainsi compilé. Prendre l'habitude de toujours compiler avec les avertissements activés est l'une des pratiques les plus rentables qu'un futur professionnel de la sécurité puisse adopter : une part importante des bugs — y compris des bugs de sécurité — sont détectables dès cette étape, avant même l'exécution du programme.

4. Premier programme

```
#include <stdio.h>

int main(void) {
    printf("Bonjour, monde !\n");
    return 0;
}
```

Ce programme minimal, bien que trivial, contient déjà tous les éléments structurels d'un programme C :

- `#include <stdio.h>` : cette directive de préprocesseur incorpore le contenu de l'en-tête `stdio.h` (*standard input/output*), qui déclare les fonctions d'entrées-sorties standard telles que `printf` et `scanf`. Sans cette inclusion, le compilateur ne connaîtrait pas la signature de `printf` et refuserait de compiler le programme (ou, selon le compilateur, émettrait un avertissement puis un comportement indéfini).
- `int main(void)` : `main` est le **point d'entrée** obligatoire de tout programme C — c'est la première fonction appelée par le système d'exploitation au lancement du programme. Le type de retour `int` indique qu'elle renvoie un entier au système d'exploitation, appelé **code de sortie** (*exit code*) : par convention, `0` signale un succès, toute autre valeur signale une erreur (souvent utilisée par des scripts pour détecter automatiquement un échec). Le mot-clé `void` entre parenthèses précise explicitement que cette version de `main` ne prend aucun paramètre (il existe une autre forme, `int main(int argc, char *argv[])`, qui permet de récupérer les arguments passés en ligne de commande, vue dans les TP).
- `printf("Bonjour, monde !\n")` : cette fonction affiche du texte formaté sur la **sortie standard** (généralement le terminal). La séquence `\n` est un caractère spécial représentant un saut de ligne.
- `return 0;` : termine l'exécution de `main` et renvoie le code de sortie `0` (succès) au système d'exploitation.

Ce programme, une fois compilé avec la commande de la section précédente puis exécuté, affiche simplement `Bonjour, monde !` suivi d'un retour à la ligne, puis se termine.

5. Structure générale d'un programme C

Au-delà de cet exemple minimal, un programme C plus réaliste suit une organisation générale récurrente, qu'il est utile de mémoriser dès le départ pour structurer correctement ses propres fichiers sources :

```
directives de préprocesseur
déclarations globales (constantes, prototypes de fonctions)
int main(void) {
    déclarations locales
    instructions
    return code;
}
définitions des autres fonctions
```

Les **directives de préprocesseur** (essentiellement des `#include`) figurent en tête de fichier, car elles doivent être traitées avant tout le reste. Viennent ensuite les **déclarations globales** : constantes partagées par tout le fichier (souvent définies avec `#define` ou `const`), et **prototypes** de fonctions — c'est-à-dire leur signature (nom, type de retour, types des paramètres) sans leur corps — qui permettent d'utiliser une fonction avant sa définition complète plus bas dans le fichier (ce mécanisme sera détaillé au chapitre 4). La fonction `main` constitue le cœur exécutable du programme, avec ses **déclarations locales** (variables utilisées uniquement dans cette fonction) et ses **instructions**. Enfin, les **définitions des autres fonctions** — le corps complet des fonctions auxiliaires — apparaissent généralement après `main`, bien que l'ordre exact importe peu tant que les prototypes ont été déclarés au préalable.

Respecter cette organisation, même sur de petits programmes, prend une importance croissante à mesure que les projets grandissent : elle rend le code prévisible à lire, ce qui facilite aussi bien la maintenance que la relecture de sécurité (*code review*).

À retenir

- Un algorithme se conçoit avant de se coder : il est défini par sa finitude, son déterminisme et son effectivité, et peut se décrire en pseudo-code indépendamment de tout langage.
- Le C est un langage compilé, statiquement typé, proche de la machine ; son absence de garde-fous automatiques en fait un excellent terrain pour comprendre l'origine des vulnérabilités mémoire.
- La chaîne de compilation (préprocesseur, compilation, édition de liens) transforme un fichier source en exécutable ; chaque étape peut révéler des erreurs différentes.
- Toujours compiler avec les avertissements activés (`-Wall -Wextra`) : la majorité des bugs de sécurité en C sont détectables dès la compilation ou l'analyse statique.

Chapitre 2 — Variables, types, opérateurs et entrées-sorties

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Variables et types de base

Une **variable** est un emplacement mémoire nommé, réservé pour stocker une valeur pendant l'exécution du programme. En C, contrairement à des langages comme Python, une variable doit être déclarée avec un **type explicite** avant d'être utilisée : ce type détermine à la fois la taille (en octets) de l'emplacement mémoire réservé, la manière dont les bits qui s'y trouvent doivent être interprétés (entier, caractère, flottant...), et l'ensemble des opérations valides sur cette variable. C'est ce qu'on appelle le **typage statique** : le type d'une variable est fixé une fois pour toutes à la compilation et ne change jamais au cours de l'exécution.

Type	Taille usuelle	Exemple	Rôle
<code>int</code>	4 octets	<code>int age = 25;</code>	entier signé
<code>unsigned int</code>	4 octets	<code>unsigned int compteur = 0;</code>	entier non signé
<code>char</code>	1 octet	<code>char lettre = 'A';</code>	caractère / petit entier
<code>float</code>	4 octets	<code>float prix = 19.99f;</code>	flottant simple précision
<code>double</code>	8 octets	<code>double pi = 3.14159265;</code>	flottant double précision
<code>long</code> , <code>short</code>	variable	<code>long total;</code>	variantes de taille de <code>int</code>

Il est essentiel de comprendre que le standard C ne garantit **pas** de taille fixe pour la plupart de ces types : il garantit uniquement des relations d'ordre entre eux (`short <= int <= long <= long long`) et une taille minimale. La taille réelle dépend de l'architecture matérielle (32 ou 64 bits) et des choix du compilateur. On ne doit donc jamais supposer, par exemple, qu'un `int` fait toujours 4 octets sur toutes les plateformes : on vérifie sa taille réelle avec l'opérateur `sizeof(type)`, évalué à la compilation. Pour les cas où une taille garantie et portable est nécessaire — ce qui est fréquent dès qu'on manipule des formats de données binaires, des protocoles réseau ou du code cryptographique — le standard C99 introduit l'entête `stdint.h`, qui fournit des types de taille fixe explicite comme `int32_t` (entier signé de 32 bits garantis), `uint8_t` (entier non signé de 8 bits, souvent utilisé pour manipuler des octets bruts) ou `uint64_t`. Cette précision n'est pas un détail académique : un calcul de taille de buffer qui suppose implicitement une taille de type erronée est une source classique de débordement d'entier (*integer overflow*), une catégorie de vulnérabilité à part entière qui sera approfondie dans le cours *Sécurité des applications*.

```
#include <stdio.h>
printf("Taille d'un int : %zu octets\n", sizeof(int));
printf("Taille d'un long : %zu octets\n", sizeof(long));
```

2. Déclaration, initialisation, portée

Déclarer une variable réserve l'espace mémoire correspondant à son type, mais ne lui donne **aucune valeur définie** tant qu'elle n'est pas explicitement initialisée. C'est une différence fondamentale avec des langages qui initialisent automatiquement les variables à une valeur par défaut (zéro, `null`, etc.) : en C, une variable locale non initialisée contient simplement ce qui se trouvait déjà à cet emplacement mémoire — une valeur imprévisible, communément appelée « valeur poubelle » (*garbage value*).

```
int x;           // déclarée, valeur indéterminée (danger !)  
int y = 0;       // déclarée et initialisée  
const int MAX = 100; // constante, non modifiable après initialisation
```

Utiliser la valeur de `x` avant de lui avoir affecté quelque chose (par exemple `printf("%d\n", x);` juste après sa déclaration) est un comportement indéfini en C : le programme peut afficher n'importe quelle valeur, différente à chaque exécution, voire un comportement encore plus surprenant selon les optimisations du compilateur. Ce défaut, anodin en apparence, est une source classique de bugs difficiles à reproduire et, dans certains contextes, une véritable vulnérabilité de sécurité : si la mémoire non initialisée provient d'une zone précédemment utilisée par une autre partie du programme (voire par un autre processus, dans certains cas d'allocation), elle peut contenir des données sensibles — mots de passe, clés, fragments de données utilisateur — qu'un attaquant pourrait exploiter pour les faire fuiter. Le mot-clé `const`, quant à lui, indique au compilateur qu'une variable ne doit plus être modifiée après son initialisation ; toute tentative de modification ultérieure (`MAX = 200;`) est détectée et rejetée à la compilation, ce qui en fait un outil simple mais efficace pour exprimer une intention et laisser le compilateur vérifier qu'elle est respectée.

Bonne pratique : initialiser systématiquement toute variable au moment de sa déclaration, même avec une valeur « neutre » (`0`, `NULL`, chaîne vide), sauf si elle est immédiatement écrasée par la suite (par exemple en tout premier paramètre de sortie d'une fonction).

3. Opérateurs

Le C fournit un ensemble d'opérateurs qui, combinés aux types vus plus haut, permettent d'exprimer tous les calculs et comparaisons nécessaires à un algorithme.

- **Arithmétiques** : `+` `-` `*` `/` effectuent respectivement addition, soustraction, multiplication et division ; `%` calcule le **reste** de la division entière (le modulo) et n'existe que pour les types entiers — l'appliquer à des `float` ou `double` est une erreur de compilation. Attention également à la division entre deux entiers : `7 / 2` vaut `3` en C (division entière, la partie décimale est tronquée), et non `3.5` ; pour obtenir un résultat flottant, il faut qu'au moins un des deux opérandes soit de type flottant (voir la section sur les conversions).
- **Relationnels** : `==` `!=` `<` `>` `<=` `>=` comparent deux valeurs et renvoient un entier valant `0` (faux) ou `1` (vrai) — en C, il n'existe pas de type booléen distinct dans le standard historique (avant `stdbool.h`, introduit en C99, qui fournit `bool`, `true`, `false` comme alias pratiques sur ce même mécanisme entier).
- **Logiques** : `&&` (ET logique), `||` (OU logique), `!` (négation) combinent des expressions booléennes. Le C applique une **évaluation en court-circuit** : dans `a() && b()`, si `a()` renvoie faux, `b()` n'est jamais évaluée, car le résultat global est déjà déterminé. Ce comportement est souvent exploité volontairement, par exemple pour vérifier qu'un pointeur n'est pas nul avant de le déréférencer : `if (p != NULL && p->valeur > 0)`.
- **Bit à bit** : `&` (ET), `|` (OU), `^` (OU exclusif, XOR), `~` (complément à un), `<<` (décalage à gauche), `>>` (décalage à droite) opèrent directement sur la représentation binaire des entiers, bit par bit. Ces opérateurs sont fondamentaux en cryptographie (le XOR, en particulier, est à la base du chiffrement de Vernam et de nombreux algorithmes symétriques), en manipulation de masques de bits (drapeaux combinés dans un seul entier) et en manipulation bas niveau de protocoles réseau ; ils seront repris en profondeur dans le chapitre *Cryptographie*.
- **Affectation composée** : `+=` `-=` `*=` `/=` `%=` combinent une opération et une affectation (`a += 5` ; équivaut à `a = a + 5` ;), ce qui rend le code plus concis et parfois plus lisible.
- **Incrémentation/décrémentation** : `++` et `--` augmentent ou diminuent une variable de 1. Utilisés en **préfixe** (`++i`), l'opération est effectuée avant que la valeur ne soit utilisée dans l'expression englobante ; en **postfixe** (`i++`), la valeur d'origine est utilisée dans l'expression, puis l'incréméntation a lieu. La différence est subtile mais peut avoir un effet observable, par exemple dans `tableau[i++] = 0` ; (utilise l'ancienne valeur de `i` comme indice, puis l'incréménte) contre `tableau[++i] = 0` ; (incréménte d'abord, puis utilise la nouvelle valeur).

```
int a = 5, b = 3;
int somme = a + b;      // 8
int reste = a % b;     // 2
int masque = a & b;    // ET bit à bit : 0101 & 0011 -> 0001, soit 1
int i = 0;
int x = tableau[i++];  // utilise tableau[0], puis i vaut 1
```

4. Conversions de type (casting)

Lorsqu'une expression mélange plusieurs types (par exemple un `int` et un `double`), le compilateur applique des règles de **conversion implicite** pour ramener les opérandes à un type commun avant d'effectuer l'opération — c'est ce qu'on appelle la **promotion** ou la **conversion arithmétique usuelle**. Le programmeur peut aussi forcer une conversion explicitement, via l'opérateur de **cast** (`type`), pour indiquer clairement son intention.

```
int i = 7;
double d = (double) i / 2;    // conversion explicite de i en double avant division : 3.5
int tronque = (int) 3.9;      // conversion explicite : 3 (troncature, pas d'arrondi)
```

Dans le premier exemple, sans le cast explicite, `i / 2` serait évalué comme une division entière (3) avant même d'être affecté à `d` — le cast est donc indispensable pour obtenir le résultat flottant attendu. Le second exemple illustre que la conversion d'un flottant vers un entier **tronque** la partie décimale (elle ne l'arrondit pas) : `(int) 3.9` vaut 3, et non 4.

Les conversions **implicites** — celles que le compilateur applique automatiquement sans que le programmeur l'ait explicitement demandé — sont une source fréquente et particulièrement insidieuse de bugs, car elles ne provoquent généralement ni erreur ni avertissement visible :

- convertir un `int` de grande valeur vers un `char` (1 octet) tronque silencieusement la valeur, ne conservant que les 8 bits de poids faible ;
- convertir un entier **signé négatif** vers un type **non signé** produit une valeur immense (par un mécanisme de représentation en complément à deux), ce qui peut fausser une comparaison ou une taille de boucle de façon désastreuse ;
- convertir une taille calculée en `size_t` (type non signé, souvent 8 octets sur une architecture 64 bits) vers un `int` (souvent 4 octets, signé) peut silencieusement tronquer ou transformer une grande valeur positive en valeur négative.

Ce dernier cas est particulièrement pertinent en sécurité : de nombreuses vulnérabilités réelles proviennent d'un calcul de taille de buffer effectué avec un type inadapté, qui « déborde » silencieusement et conduit ensuite à une allocation ou une copie de taille incorrecte. La vigilance sur les types utilisés dans les calculs de tailles est donc une compétence de sécurité à part entière, pas un simple détail de style.

5. Entrées-sorties standard

Les fonctions `printf` (affichage) et `scanf` (saisie), déclarées dans `stdio.h`, constituent l'interface la plus courante entre un programme C et l'utilisateur en mode texte.

```
#include <stdio.h>

int main(void) {
    int age;
    printf("Entrez votre âge : ");
    scanf("%d", &age);          // & : adresse de la variable
    printf("Vous avez %d ans.\n", age);
    return 0;
}
```

Les deux fonctions s'appuient sur une **chaîne de format** contenant des **spécificateurs**, précédés du symbole `%`, qui indiquent le type de la valeur à lire ou à écrire à cet endroit précis : `%d` pour un `int`, `%f` pour un `float` ou un `double` (en affichage), `%c` pour un `char`, `%s` pour une chaîne de caractères, `%p` pour afficher une adresse mémoire (un pointeur), `%x` pour afficher un entier en notation hexadécimale. Il est crucial que le spécificateur corresponde exactement au type de la variable fournie : un mauvais spécificateur (par exemple `%d` sur un `double`) est un comportement indéfini que le compilateur ne détecte pas toujours par défaut, mais que `-Wall -Wextra` (chapitre 1) signale généralement.

Point notable dans l'exemple : `scanf("%d", &age)` prend `&age`, l'**adresse** de la variable `age`, et non `age` elle-même. C'est parce que `scanf` doit pouvoir **modifier** la variable de l'appelant pour y écrire la valeur saisie — un mécanisme de passage par adresse qui sera formalisé au chapitre 4 et qui repose directement sur les pointeurs, introduits au chapitre 6. Oublier le `&` (une erreur très fréquente chez les débutants) provoque un comportement indéfini : `scanf` interprète alors la valeur de `age` elle-même comme une adresse mémoire et tente d'y écrire, ce qui peut planter le programme ou, plus insidieusement, corrompre une zone mémoire arbitraire.

Point de vigilance sécurité : `scanf("%s", buffer)` sans limite de taille explicite continue de lire des caractères tant qu'il n'y a pas d'espace, sans jamais vérifier que `buffer` est assez grand pour les contenir — une saisie trop longue provoque alors un débordement de tampon, exactement comme `strcpy` (détaillé au chapitre 5). De la même façon, transmettre directement une chaîne contrôlée par l'utilisateur comme chaîne de format à `printf` (par exemple `printf(saisie_utilisateur);` au lieu de `printf("%s", saisie_utilisateur);`) constitue une vulnérabilité de type **format string** : un attaquant peut y injecter ses propres spécificateurs (`%x`, `%n`...) pour lire, voire écrire, dans la mémoire du programme. Ces deux points seront repris et exploités en détail dans le cours *Sécurité des applications*, mais il est important d'en avoir conscience dès l'apprentissage des bases du langage.

À retenir

- Toujours initialiser ses variables : une variable non initialisée contient une valeur imprévisible, source de bugs et de fuites potentielles de données sensibles.
- Connaître la taille et les limites de chaque type (`limits.h` pour les bornes, `stdint.h` pour des tailles garanties) évite les débordements d'entiers silencieux.
- Se méfier des conversions implicites, en particulier entre types signés/non signés et entre tailles différentes : elles ne provoquent aucune erreur visible mais peuvent fausser un calcul critique.
- Ne jamais utiliser une entrée utilisateur comme chaîne de format de `printf`, et toujours borner la taille des saisies avec `scanf` ou ses alternatives plus sûres.

Chapitre 3 — Structures de contrôle

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU


```

printf("Très bien\n");
} else if (note >= 10) {
    printf("Admis\n");
} else {
    printf("Ajourné\n");
}

```

Le C évalue les conditions **dans l'ordre d'écriture** et exécute le premier bloc dont la condition est vraie, en ignorant tous les suivants : dans l'exemple ci-dessus, si `note` vaut 18, seule la première condition (`note >= 16`) est testée et son bloc exécuté ; les blocs suivants ne sont même pas évalués. Il est important de noter qu'en C, toute expression entière non nulle est considérée comme « vraie » et la valeur 0 comme « fausse » — ce qui permet d'écrire `if (compteur)` comme raccourci de `if (compteur != 0)`, une pratique courante mais qu'il faut savoir lire pour ne pas la confondre avec une simple faute de frappe (`if (compteur = 0)` avec un seul `=` est une **erreur d'affectation**, syntaxiquement valide mais presque toujours un bug, que certains compilateurs signalent avec `-Wall`).

switch

```

int jour = 3;
switch (jour) {
    case 1: printf("Lundi\n"); break;
    case 2: printf("Mardi\n"); break;
    case 3: printf("Mercredi\n"); break;
    default: printf("Jour invalide\n");
}

```

La structure `switch` compare une expression (ici `jour`) à une série de valeurs constantes (les `case`), et exécute les instructions du `case` correspondant. Contrairement à un enchaînement de `if / else if`, l'exécution ne s'arrête pas automatiquement à la fin d'un bloc `case` : sans instruction `break`, le flux d'exécution continue vers le `case` suivant, quelle que soit sa valeur — un comportement appelé **fall-through** (« chute au travers »). C'est parfois voulu (par exemple pour regrouper plusieurs valeurs traitées de façon identique : `case 6: case 7: printf("Week-end\n"); break;`), mais c'est très souvent une source de bug lorsqu'il est involontaire, notamment lorsqu'un `case` est ajouté a posteriori sans que le développeur ne pense à ajouter le `break` correspondant. La clause `default`, optionnelle mais fortement recommandée, capture toutes les valeurs non prévues explicitement par un `case` — l'omettre revient à ignorer silencieusement les cas non anticipés, ce qui peut masquer une erreur logique.

Bonne pratique : documenter systématiquement par un commentaire tout `fall-through` intentionnel (par exemple `// fall through volontaire`),

while

```
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

La boucle `while` teste sa condition **avant** chaque itération, y compris la toute première : si la condition est fausse dès le départ, le corps de la boucle n'est jamais exécuté. Elle convient particulièrement bien lorsque le nombre d'itérations n'est pas connu à l'avance et dépend d'un état qui évolue au fil de l'exécution (par exemple, lire des données tant qu'il en reste, ou rechercher un élément tant qu'il n'est pas trouvé).

do ... while

```
int choix;
do {
    printf("Choix (1-3) : ");
    scanf("%d", &choix);
} while (choix < 1 || choix > 3);
```

La boucle `do ... while` inverse l'ordre : elle exécute d'abord le corps de la boucle, puis teste la condition — garantissant ainsi que le corps s'exécute **au moins une fois**, quelle que soit la condition initiale. Cette structure est particulièrement adaptée à la validation d'une saisie utilisateur : on a besoin de demander la valeur au moins une fois avant de pouvoir la tester, ce qu'une boucle `while` classique rendrait plus maladroit à exprimer (il faudrait initialiser artificiellement la variable de condition avant la boucle).

for

```
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
}
```

3. Rupture de séquence

En complément des trois structures de boucle, le C fournit des instructions permettant de modifier explicitement le déroulement normal d'une boucle ou d'un `switch` :

- **break** : interrompt immédiatement la boucle ou le `switch` englobant et poursuit l'exécution juste après. Dans une boucle imbriquée, `break` ne sort que de la boucle la plus interne — il n'existe pas de mécanisme natif en C pour sortir directement de plusieurs niveaux de boucles imbriquées avec `break` seul.
- **continue** : interrompt l'itération courante et passe directement à l'itération suivante, en réévaluant la condition de la boucle (et, dans le cas d'un `for`, en exécutant d'abord l'étape d'incrément). C'est utile pour « sauter » certains cas particuliers sans imbriquer davantage de conditions dans le corps de la boucle.
- **goto** : transfère le contrôle de façon inconditionnelle vers une étiquette (*label*) définie ailleurs dans la même fonction. Son usage est **fortement déconseillé** dans la très grande majorité des cas, car il nuit gravement à la lisibilité et à la prévisibilité du flux d'exécution (« code spaghetti »). Il subsiste néanmoins des cas d'usage reconnus et acceptés, notamment pour sortir proprement d'imbrications profondes de boucles, ou pour implémenter un mécanisme de nettoyage centralisé en cas d'erreur (motif dit de « *goto cleanup* »), fréquent dans du code C bas niveau — noyau Linux, pilotes — où il est utilisé de façon disciplinée et systématique plutôt qu'ad hoc.

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) break;          // arrête complètement la boucle à i == 5  
    if (i % 2 == 0) continue;  // saute les valeurs paires, sans les afficher  
    printf("%d ", i);  
}
```

4. Boucles infinies et conditions de terminaison

```
for (;;) {  
    // boucle infinie volontaire, sortie via break  
}
```

Cette écriture, `for (;;)`, omet les trois éléments habituels du `for` : sans condition explicite, elle est toujours considérée comme vraie, ce qui produit une boucle qui ne s'arrête jamais d'elle-même. C'est un idiome courant et parfaitement valide en C — par exemple pour la boucle principale d'un serveur qui doit tourner indéfiniment — à condition qu'une instruction `break` (ou un `return`, ou un `exit`) quelque part dans le corps de la boucle permette effectivement d'en sortir dans les conditions voulues.

Le vrai danger ne réside donc pas dans l'écriture explicite d'une boucle infinie volontaire, mais dans une boucle censée se terminer et qui, à cause d'une erreur de logique, ne le fait jamais : un compteur qui n'est jamais incrémenté, une condition de sortie qui ne peut structurellement jamais devenir vraie, ou une variable de contrôle modifiée par erreur à l'intérieur de la boucle. Une telle boucle consomme indéfiniment des ressources (temps processeur, éventuellement mémoire si elle alloue à chaque itération) sans jamais rendre la main : dans un service exposé, c'est un vecteur classique de **déni de service** (*denial of service*), qu'une entrée malveillante suffisamment spécifique peut parfois déclencher délibérément. Cette classe de problème sera reprise sous un angle plus large dans le cours *Security by design*.

Bonne pratique : pour toute boucle `while` ou `do while`, vérifier explicitement, à l'écriture du code, que chaque chemin d'exécution possible dans le corps de la boucle fait progresser la condition vers sa sortie.

5. Complexité algorithmique (introduction)

Le nombre d'itérations effectuées par les boucles d'un algorithme, en fonction de la taille des données qu'il traite, détermine ce qu'on appelle sa **complexité temporelle** : une mesure, indépendante du matériel utilisé, de la façon dont le temps d'exécution croît lorsque la taille des données d'entrée augmente. Cette croissance s'exprime en **notation de Landau** (le grand O), qui décrit le comportement asymptotique — c'est-à-dire pour des données de grande taille — en ignorant les facteurs constants et les termes d'ordre inférieur.

Deux exemples illustrent concrètement cette notion, tous deux détaillés et implémentés en TP :

- Une **recherche séquentielle** d'un élément dans un tableau non trié de taille n doit, dans le pire cas, examiner chacun des n éléments un par un : sa complexité est notée **$O(n)$** , elle croît linéairement avec la taille des données.
- Une **recherche dichotomique** (ou recherche par bisection) dans un tableau **trié** élimine, à chaque comparaison, la moitié des éléments restants encore candidats : sa complexité est notée **$O(\log n)$** , une croissance beaucoup plus lente — pour un tableau d'un million d'éléments, une recherche dichotomique nécessite au plus une vingtaine de comparaisons, contre potentiellement un million pour une recherche séquentielle.

Cette différence n'est pas une nuance purement théorique : elle a des conséquences pratiques directes sur la performance d'un programme réel dès que les volumes de données deviennent significatifs. La notion de complexité sera réutilisée à plusieurs reprises plus loin dans le cursus, en particulier pour évaluer le coût de calcul de certains algorithmes cryptographiques (chapitre *Cryptographie*) et pour estimer, en ordre de grandeur, le nombre d'opérations nécessaires à une attaque par force brute (chapitre *Cryptanalyse*) — la sécurité de nombreux mécanismes cryptographiques repose précisément sur le fait que la meilleure attaque connue a une complexité si élevée qu'elle est irréalisable en pratique avec les ressources de calcul actuelles.

À retenir

- Choisir la structure adaptée : `for` pour un nombre d'itérations connu à l'avance, `while` pour une condition testée avant chaque itération, `do while` lorsqu'il faut exécuter le corps au moins une fois.
- Toujours s'assurer qu'une boucle progresse effectivement vers sa condition d'arrêt, sous peine de boucle infinie non voulue et de déni de service potentiel.
- `switch` sans `break` provoque un fall-through vers le `case` suivant : un piège classique à documenter explicitement lorsqu'il est intentionnel.
- La complexité algorithmique (notation grand O) mesure la croissance du temps d'exécution avec la taille des données ; elle sera réutilisée pour analyser des algorithmes cryptographiques et des attaques par force brute.

Chapitre 4 — Fonctions et portée des variables

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Pourquoi découper en fonctions ?

Une **fonction** encapsule un traitement réutilisable sous un nom : elle prend éventuellement des données en entrée (les **paramètres**), effectue un calcul ou une action, et produit éventuellement un résultat en sortie (la **valeur de retour**). Découper un programme en fonctions plutôt que d'écrire l'intégralité de la logique dans `main` répond à plusieurs objectifs complémentaires, qui deviennent de plus en plus déterminants à mesure qu'un projet grandit :

- **Réutilisabilité** : un traitement écrit une seule fois sous forme de fonction peut être appelé autant de fois que nécessaire, à différents endroits du programme, sans dupliquer le code — ce qui réduit les erreurs (une correction n'est à faire qu'à un seul endroit) et la taille du code source.
- **Lisibilité** : un `main` qui appelle des fonctions bien nommées (`valider_saisie()`, `calculer_moyenne()`, `afficher_resultat()`) se lit presque comme une description de haut niveau de l'algorithme, indépendamment des détails d'implémentation de chaque étape.
- **Testabilité** : une fonction isolée, avec des entrées et une sortie clairement définies, peut être testée indépendamment du reste du programme — un principe fondamental des tests unitaires, qui deviennent nettement plus difficiles à écrire lorsque toute la logique est enchevêtrée dans une seule grande fonction.
- **Limitation de la portée des erreurs** : si une fonction est petite et se limite à une seule responsabilité clairement identifiée, un bug qu'elle contient est plus facile à localiser, à comprendre et à corriger qu'un bug perdu au milieu de centaines de lignes mêlant plusieurs préoccupations différentes.

Ce dernier point est directement lié à l'un des principes fondateurs du *Security by design* : plus une fonction est petite, ciblée et prévisible, plus elle est facile à auditer — à relire méthodiquement pour vérifier qu'elle ne contient pas de faille — et plus il est facile de raisonner formellement sur ce qu'elle garantit (ses préconditions, ses postconditions) sans avoir à tenir compte de l'état de tout le reste du programme.

2. Déclaration, prototype, définition

```
// Prototype (souvent dans un .h)
int addition(int a, int b);

int main(void) {
    int resultat = addition(3, 4);
    printf("%d\n", resultat);
    return 0;
}

// Définition
int addition(int a, int b) {
    return a + b;
}
```

Il faut distinguer deux notions souvent confondues par les débutants : le **prototype** et la **définition** d'une fonction. Le prototype (`int addition(int a, int b);`) déclare uniquement la **signature** de la fonction — son nom, le type de la valeur qu'elle renvoie, et le type (et éventuellement le nom) de chacun de ses paramètres — sans fournir son corps. La définition (`int addition(int a, int b) { return a + b; }`) fournit, elle, l'implémentation complète.

Le compilateur C lit un fichier source de façon séquentielle, de haut en bas : il ne peut vérifier un appel de fonction (`addition(3, 4)`) que s'il a déjà rencontré, avant cet appel, soit la définition complète de la fonction, soit au minimum son prototype. Déclarer le prototype en tête de fichier (ou, plus couramment dans un projet réel, dans un fichier d'en-tête `.h` inclus par `#include`) permet donc d'appeler une fonction avant sa définition complète plus bas dans le fichier, tout en permettant au compilateur de vérifier immédiatement la cohérence des types passés à chaque appel : si `addition` est appelée avec un mauvais nombre d'arguments, ou avec un type incompatible, le compilateur le détecte et signale une erreur — une vérification précieuse qu'une absence totale de prototype (pratique tolérée par d'anciens standards C, mais fortement déconseillée) laisserait passer silencieusement, avec un comportement indéfini à l'exécution.

3. Passage de paramètres

Le C connaît deux modes de passage de paramètres à une fonction, dont la distinction est essentielle à comprendre pour prévoir correctement ce qu'une fonction peut ou ne peut pas modifier chez son appelant.

- **Passage par valeur** (comportement par défaut en C, pour tous les types de base) : lorsqu'une fonction est appelée, chaque paramètre reçoit une **copie** de la valeur fournie par l'appelant. Toute modification effectuée sur ce paramètre à l'intérieur de la fonction n'affecte que cette copie locale, et disparaît dès que la fonction se termine — la variable d'origine, du côté de l'appelant, reste totalement inchangée.
- **Passage par adresse** (via un pointeur) : plutôt que de transmettre une copie de la valeur, on transmet l'**adresse mémoire** de la variable de l'appelant. La fonction reçoit alors un pointeur vers cette variable et peut, en le déréférençant, lire et **modifier directement** la valeur d'origine.

```
void incrementer(int *x) {
    (*x)++;
}

int main(void) {
    int n = 5;
    incrementer(&n); // n vaut 6 après l'appel
    return 0;
}
```

Dans cet exemple, `incrementer` reçoit non pas une copie de `n`, mais l'adresse de `n` (obtenue avec `&n` lors de l'appel). À l'intérieur de la fonction, `x` est un pointeur vers cette adresse ; l'expression `(*x)++` déréférence `x` pour accéder à la variable pointée (`n`) et l'incrmente directement. C'est le **seul** mécanisme dont dispose le C pour permettre à une fonction de modifier une variable de l'appelant : sans passage par adresse, une fonction telle que `void incrementer(int x) { x++; }` compilerait parfaitement, mais n'aurait strictement aucun effet visible en dehors d'elle-même, car elle n'incrémenterait qu'une copie locale immédiatement perdue à son retour. Ce mécanisme sera repris et approfondi dans le chapitre sur les pointeurs (chapitre 6), où l'on verra qu'il est également au cœur du passage de tableaux à des fonctions.

Deux notions distinctes, mais souvent confondues, décrivent le cycle de vie d'une variable : sa **portée** (scope — dans quelle partie du code son nom est-il visible et utilisable) et sa **durée de vie** (*lifetime* — combien de temps l'emplacement mémoire qui lui est associé reste-t-il valide).

Type de variable	Portée	Durée de vie
locale	bloc où elle est déclarée	jusqu'à la fin du bloc (pile)
globale	tout le fichier (ou programme si non <code>static</code>)	toute l'exécution
<code>static</code> locale	bloc de déclaration	toute l'exécution, valeur conservée entre appels
paramètre	corps de la fonction	durée de l'appel

Une **variable locale**, déclarée à l'intérieur d'une fonction ou d'un bloc `{ }`, n'est visible et utilisable qu'à l'intérieur de ce bloc ; elle est automatiquement créée sur la **pile** (*stack*, voir chapitre 6) à l'entrée du bloc et détruite à sa sortie — toute tentative d'y accéder en dehors de son bloc est une erreur de compilation. Une **variable globale**, déclarée en dehors de toute fonction, est visible dans tout le reste du fichier (et dans d'autres fichiers du même programme si elle n'est pas marquée `static`) et existe pendant toute la durée de vie du programme.

Le cas de la variable **`static` locale** mérite une attention particulière, car il combine une portée limitée (comme une variable locale ordinaire) avec une durée de vie étendue (comme une variable globale) :

```
int compteur_appels(void) {
    static int n = 0; // initialisée une seule fois, à la première exécution de cette ligne
    n++;
    return n;
}
```

Contrairement à une variable locale ordinaire, `n` n'est **pas** recrée et réinitialisée à chaque appel de `compteur_appels` : elle est initialisée une seule fois, lors du tout premier appel, puis conserve sa valeur d'un appel à l'autre pendant toute la durée de vie du programme — un appel successif de `compteur_appels()` renverra donc 1, puis 2, puis 3, et ainsi de suite. Ce mécanisme est pratique pour maintenir un état discret entre plusieurs appels (un compteur, un cache simple) sans recourir à une variable globale visible de tout le fichier, mais il introduit un **état caché** dans la fonction : deux appels successifs à `compteur_appels()`, avec exactement les mêmes arguments (ici, aucun), ne produisent plus le même résultat, ce qui rend la fonction plus difficile à raisonner, à tester isolément et à paralléliser en toute sécurité.

Plus largement, les **variables globales** facilitent le partage d'état entre différentes parties d'un programme sans avoir à le transmettre explicitement en paramètre, mais ce confort a un coût : elles nuisent à la prévisibilité du code (n'importe quelle fonction du programme peut potentiellement les modifier, ce qui rend difficile de savoir, en lisant une seule fonction, d'où vient une valeur inattendue) et compliquent considérablement l'analyse de

5. Récursivité

```
unsigned long factorielle(unsigned int n) {  
    if (n == 0) return 1;  
    return n * factorielle(n - 1);  
}
```

Une fonction **réursive** est une fonction qui s'appelle elle-même, directement ou indirectement (via une autre fonction), pour résoudre un problème en le ramenant à une version plus petite de lui-même. Toute fonction récursive correctement conçue comporte deux éléments indispensables : un **cas de base** (ici, `n == 0`, qui renvoie directement un résultat sans nouvel appel récursif) qui arrête la récursion, et un **cas récursif** (ici, `n * factorielle(n - 1)`) qui rapproche progressivement le problème du cas de base à chaque appel. Sans cas de base atteignable, ou si le cas récursif ne se rapproche jamais réellement du cas de base, la récursion ne s'arrête jamais — l'équivalent, pour une fonction, d'une boucle infinie.

Le mécanisme sous-jacent qui rend la récursivité possible est la **pile d'appels** : chaque appel de fonction, récursif ou non, réserve un espace sur la pile (une *stack frame*) pour stocker ses variables locales, ses paramètres et l'adresse de retour vers l'appelant. Dans un appel récursif, chaque nouvel appel empile un nouveau cadre par-dessus les précédents, qui restent tous en mémoire tant que l'appel correspondant n'est pas terminé. Une récursion **non bornée**, ou simplement trop profonde par rapport à la taille de pile allouée au programme (souvent de l'ordre de quelques mégaoctets par défaut), épuise cet espace et provoque un **stack overflow** (débordement de pile) — un cas concret, et facile à reproduire expérimentalement, de vulnérabilité par épuisement de ressource : si la profondeur de récursion dépend d'une entrée fournie par un utilisateur (par exemple, une donnée dont la taille contrôle indirectement le nombre d'appels récursifs), un attaquant peut potentiellement provoquer volontairement ce débordement pour faire planter le programme, voire, dans certaines conditions, en exploiter les conséquences mémoire.

6. Bonnes pratiques

- **Une fonction, une responsabilité claire** : si le nom d'une fonction nécessite un « et » pour être décrit précisément (« valide et enregistre l'utilisateur »), c'est souvent le signe qu'elle devrait être scindée en plusieurs fonctions plus ciblées.
- **Toujours valider les paramètres reçus** : vérifier qu'un pointeur passé en paramètre n'est pas `NULL` avant de le déréférencer, vérifier qu'une taille est cohérente (positive, dans les bornes attendues) avant de l'utiliser pour une allocation ou un accès à un tableau. Une fonction qui fait confiance aveuglément à ses paramètres, sans les valider, propage silencieusement les erreurs de son appelant au lieu de les détecter tôt.
- **Documenter précondition et postcondition** : préciser explicitement, en commentaire ou dans la documentation associée, ce que la fonction attend en entrée (précondition — par exemple « `n` doit être positif ou nul ») et ce qu'elle garantit en sortie (postcondition), y compris — point souvent oublié et pourtant crucial pour éviter fuites mémoire et double libérations — qui, de l'appelant ou de la fonction, est responsable de libérer une éventuelle mémoire allouée dynamiquement par la fonction ou transmise à travers elle.

À retenir

- Le passage par adresse (via un pointeur) est l'unique mécanisme du C permettant à une fonction de modifier une variable de l'appelant ; le passage par valeur, par défaut, ne modifie qu'une copie locale.
- La portée `static` locale crée un état persistant discret entre les appels d'une fonction : pratique mais à utiliser avec parcimonie, car elle nuit à la prévisibilité et à la testabilité.
- Les variables globales facilitent le partage d'état mais introduisent des effets de bord non locaux qui compliquent l'analyse de sécurité : à limiter au strict nécessaire.
- La récursivité est élégante pour exprimer certains algorithmes, mais reste bornée par la taille de la pile ; une récursion non maîtrisée est une vulnérabilité par épuisement de ressource.

Chapitre 5 — Tableaux et chaînes de caractères

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Tableaux statiques

```
int notes[5] = {12, 15, 8, 17, 10};
int i;
for (i = 0; i < 5; i++) {
    printf("%d\n", notes[i]);
}
```

Un tableau en C, tel que `notes[5]`, réserve un **bloc contigu** de mémoire capable de contenir exactement cinq entiers, alloué en une seule fois au moment de sa déclaration. Cette contiguïté n'est pas un détail d'implémentation anodin : elle explique directement pourquoi l'indexation `notes[i]` fonctionne, et surtout ce qu'elle fait réellement « sous le capot ». En C, `notes[i]` est **strictement équivalent**, par définition du langage, à l'expression `*(notes + i)` : le nom du tableau, utilisé dans une expression, se comporte comme un pointeur vers son premier élément (`notes[0]`), et `notes + i` calcule l'adresse de l'élément situé `i` positions plus loin (l'arithmétique de pointeurs, détaillée au chapitre 6, tient automatiquement compte de la taille du type — ici `sizeof(int)` — pour ce calcul). Accéder à `notes[i]`, c'est donc calculer une adresse mémoire puis y lire ou écrire une valeur, rien de plus.

Cette équivalence a une conséquence fondamentale, à retenir dès maintenant : **le C n'effectue aucune vérification de borne à l'exécution**. Contrairement à des langages qui interrompent immédiatement le programme (en levant une exception) dès qu'un indice sort des limites valides d'un tableau, le C calcule simplement l'adresse correspondante et accède à la mémoire qui s'y trouve — que cette adresse appartienne réellement au tableau ou non. Ainsi, lire ou écrire `notes[5]` (un indice hors limites, puisque les indices valides vont de 0 à 4) ou `notes[-1]` **compile sans erreur et s'exécute**, en accédant à de la mémoire adjacente au tableau, dont le contenu et la structure sont totalement imprévisibles du point de vue du programme. C'est la cause première d'une des vulnérabilités logicielles les plus exploitées historiquement, le **débordement de tampon** (*buffer overflow*) : lorsque l'indice ou la quantité de données écrites dans un tableau dépend, directement ou indirectement, d'une entrée fournie par un utilisateur non fiable, un attaquant peut délibérément provoquer une écriture hors limites pour corrompre des données adjacentes, voire, dans les cas les plus graves, détourner le flux d'exécution du programme. Ce sujet sera approfondi et mis en pratique dans le cours *Sécurité des applications*.

2. Tableaux à deux dimensions

```
int matrice[3][4];  
matrice[1][2] = 7;
```

Un tableau à deux dimensions, tel que `matrice[3][4]` (3 lignes, 4 colonnes), n'est en réalité, en mémoire, qu'un unique bloc contigu de $3 \times 4 = 12$ entiers : le C ne dispose pas d'une structure de données « tableau 2D » distincte, il s'agit simplement d'un tableau d'entiers organisé et adressé selon deux indices au lieu d'un. Le stockage se fait en **row-major order** (ordre ligne par ligne) : tous les éléments d'une même ligne sont contigus en mémoire avant que ne commence la ligne suivante. Concrètement, l'élément `matrice[1][2]` se trouve, en mémoire, à la position $1 * 4 + 2 = 6$ en partant du début du bloc — un calcul que le compilateur effectue automatiquement à chaque accès, mais qu'il est utile de comprendre pour raisonner sur la performance (parcourir une matrice ligne par ligne, dans l'ordre naturel de stockage, est nettement plus efficace pour le cache processeur que de la parcourir colonne par colonne) et pour comprendre pourquoi, comme pour un tableau à une dimension, aucune vérification de borne n'est effectuée sur l'un ou l'autre des deux indices.

Ici, `nom` est un tableau capable de contenir 20 caractères, mais seuls les 7 premiers sont occupés par "Zakaria", suivis automatiquement d'un `'\0'` (ajouté implicitement par le compilateur lors de cette initialisation littérale), le reste du tableau restant simplement disponible et non initialisé. La fonction `strlen` ne renvoie **pas** la taille du tableau (qui est de 20 octets) mais le nombre de caractères **avant** le `'\0'`, qu'elle détermine en parcourant la chaîne caractère par caractère jusqu'à trouver cet octet nul — une distinction essentielle entre la **taille allouée** d'un buffer et la **longueur** de la chaîne qu'il contient actuellement.

Fonctions de `string.h`

Fonction	Rôle	Risque
<code>strlen</code>	longueur d'une chaîne	boucle infinie (ou lecture hors limites) si pas de <code>'\0'</code> présent dans le buffer
<code>strcpy(dst, src)</code>	copie une chaîne	aucune vérification de taille : écrit autant d'octets que nécessaire, même si <code>dst</code> est trop petit → débordement
<code>strncpy(dst, src, n)</code>	copie bornée à <code>n</code> octets	ne garantit pas la terminaison par <code>'\0'</code> si <code>src</code> fait <code>n</code> octets ou plus
<code>strcat(dst, src)</code>	concaténation	même risque que <code>strcpy</code> : aucune vérification que <code>dst</code> a assez de place pour accueillir le résultat
<code>strcmp(a, b)</code>	comparaison lexicographique (0 si égales)	—
<code>snprintf</code>	formatage borné dans un buffer	alternative sûre recommandée à <code>sprintf</code> , <code>strcpy</code> et <code>strcat</code>

Ce tableau met en évidence un point essentiel : la plupart des fonctions historiques de manipulation de chaînes en C ont été conçues à une époque où la sécurité n'était pas une préoccupation de conception première, et **aucune d'entre elles ne connaît la taille réelle du buffer de destination** — c'est entièrement au programmeur de s'assurer, avant chaque appel, que ce buffer est suffisamment grand pour accueillir le résultat.

```
char buffer[10];
strcpy(buffer, "Ceci dépasse dix caractères"); // comportement indéfini : écrase la mémoire adjacente
```

Dans cet exemple, la chaîne source fait 29 caractères (plus le `'\0'`), largement plus que les 10 octets disponibles dans `buffer`. `strcpy` ne s'en aperçoit à aucun moment : elle copie caractère par caractère jusqu'au `'\0'` de la source, sans jamais vérifier qu'elle dépasse la taille de `buffer`. Le résultat est un **comportement indéfini** : les octets excédentaires sont écrits dans la mémoire qui suit immédiatement `buffer`, corrompant potentiellement d'autres variables, des structures de contrôle internes, voire — dans le cas d'un tableau déclaré sur la pile — l'adresse de retour de

4. Tableaux de chaînes

```
const char *jours[] = {"Lundi", "Mardi", "Mercredi"};
printf("%s\n", jours[1]); // Mardi
```

Ici, `jours` n'est pas un tableau de caractères mais un **tableau de pointeurs vers des chaînes de caractères** : chaque élément `jours[i]` est un `const char *`, c'est-à-dire l'adresse du premier caractère d'une chaîne littérale stockée ailleurs en mémoire (typiquement dans une zone en lecture seule du programme). Le mot-clé `const` indique que ces chaînes ne doivent pas être modifiées par le programme — tenter de le faire (par exemple `jours[0][0] = 'X';`) est un comportement indéfini, car les littéraux de chaîne en C sont généralement placés dans une région mémoire protégée en écriture. Cette structure — un tableau de pointeurs, chacun désignant une chaîne indépendante, potentiellement de longueurs différentes — est très répandue en C, notamment pour représenter des listes de mots-clés, de messages, ou encore les arguments de la ligne de commande (`argv`, vu en TP).

5. Parcours et algorithmes classiques

L'étude des tableaux est indissociable de quelques algorithmes fondamentaux qui les parcourent et les manipulent, et qui reviendront constamment tout au long du cursus, y compris dans des contextes de sécurité :

- **Recherche** d'un élément : la **recherche séquentielle**, qui compare l'élément recherché à chaque case du tableau dans l'ordre, fonctionne sur n'importe quel tableau mais a une complexité $O(n)$ (chapitre 3) ; la **recherche dichotomique**, applicable uniquement sur un tableau préalablement **trié**, élimine la moitié des candidats restants à chaque comparaison et atteint une complexité $O(\log n)$, bien plus efficace pour de grands volumes de données.
- **Tri** : les algorithmes de tri par **sélection**, par **insertion** et à **bulles** constituent une introduction pédagogique aux algorithmes de tri — simples à comprendre et à implémenter, mais de complexité $O(n^2)$, donc peu adaptés à de très grands tableaux. Ils seront complétés en travaux pratiques par des algorithmes de tri plus efficaces, comme le **tri rapide** (*quicksort*) ou le **tri fusion** (*merge sort*), tous deux de complexité moyenne $O(n \log n)$.
- **Inversion d'un tableau en place** : technique dite « des deux pointeurs » (ou deux indices), qui échange l'élément situé au début du tableau avec celui situé à la fin, puis progresse simultanément vers le centre jusqu'à ce que les deux indices se croisent — sans nécessiter de tableau auxiliaire. Cette même technique de parcours par deux extrémités convergentes se retrouve, sous une forme différente, en cryptographie lors de la manipulation de l'ordre des octets (*endianness*, l'ordre dans lequel les octets d'une valeur multi-octets sont stockés en mémoire) et du *padding* (ajout d'octets de remplissage pour atteindre une taille de bloc requise), deux notions reprises dans le chapitre *Cryptographie*.

```
void inverser(int tab[], int taille) {
    int debut = 0, fin = taille - 1;
    while (debut < fin) {
        int temp = tab[debut];
        tab[debut] = tab[fin];
        tab[fin] = temp;
        debut++;
        fin--;
    }
}
```

À retenir

- Un tableau C ne connaît pas sa propre taille à l'exécution : il faut systématiquement la transmettre séparément à toute fonction qui le manipule.
- Le C n'effectue aucune vérification de borne : `notes[i]` n'est qu'un calcul d'adresse (`*(notes + i)`), et la responsabilité de la validité des indices incombe entièrement au programmeur.
- Une chaîne C est un tableau de `char` terminé par `'\0'` ; toute fonction qui manipule des chaînes sans vérifier explicitement les tailles de buffer (`strcpy`, `strcat`, `scanf("%s", ...)`) est un point d'attention sécurité majeur.
- Préférer systématiquement les alternatives bornées (`snprintf`, vérification manuelle des tailles) aux fonctions historiques non sécurisées.

Chapitre 6 — Pointeurs et gestion mémoire

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Qu'est-ce qu'un pointeur ?

Un **pointeur** est une variable comme une autre, à une différence près : au lieu de stocker directement une donnée (un entier, un caractère...), elle stocke une **adresse mémoire** — c'est-à-dire l'emplacement où se trouve une autre donnée. Cette indirection est l'un des concepts les plus structurants du langage C, et probablement le plus déroutant au premier abord : comprendre les pointeurs, c'est comprendre que toute variable, quelle qu'elle soit, occupe une adresse précise en mémoire, et qu'on peut manipuler cette adresse elle-même comme une valeur à part entière.

```
int x = 10;
int *p = &x;      // p contient l'adresse de x
printf("%d\n", *p); // déréférencement : affiche 10
*p = 20;          // modifie x via le pointeur : x vaut 20
```

Deux opérateurs, symétriques l'un de l'autre, permettent de passer d'une variable à son adresse et inversement :

- **&x** : l'opérateur « adresse de » (*address-of*) renvoie l'adresse mémoire où est stockée la variable **x**. C'est cette adresse qui est ensuite stockée dans le pointeur **p**.
- ***p** : l'opérateur de **déréférencement** (*dereference*) part de l'adresse contenue dans **p** et accède à la valeur qui s'y trouve réellement — c'est-à-dire, dans cet exemple, la valeur de **x**. Utilisé à gauche d'une affectation (***p = 20;**), il permet non seulement de lire mais aussi de **modifier** la variable pointée, exactement comme si on écrivait **x = 20;** directement.

Le type du pointeur (**int *p**) importe : il indique au compilateur combien d'octets lire ou écrire à l'adresse pointée à chaque déréférencement, et permet de détecter à la compilation des erreurs de type (par exemple assigner par erreur l'adresse d'un **double** à un pointeur de type **int ***). Un pointeur non initialisé, ou initialisé à une adresse invalide, est extrêmement dangereux : le déréférencer accède à une zone mémoire arbitraire, avec un comportement totalement imprévisible.

2. Les quatre zones de la mémoire d'un programme

Pour comprendre pleinement ce que fait un pointeur et où vivent réellement les données qu'il désigne, il faut avoir une vue d'ensemble de la façon dont un programme en cours d'exécution organise sa mémoire. Cette organisation se décompose classiquement en quatre grandes zones :

Zone	Contenu	Durée de vie
Code (text)	instructions du programme, compilées en langage machine	statique, en lecture seule pendant toute l'exécution
Données statiques	variables globales et variables <code>static</code>	toute l'exécution du programme
Tas (heap)	allocation dynamique (<code>malloc</code> , <code>calloc</code> , <code>realloc</code>)	depuis l'allocation jusqu'à l'appel explicite de <code>free</code>
Pile (stack)	variables locales, paramètres de fonctions, adresses de retour	durée du bloc ou de l'appel de fonction correspondant

La zone **code** contient les instructions machine du programme lui-même, généralement protégée en écriture — un programme ne peut normalement pas modifier ses propres instructions pendant son exécution, ce qui constitue d'ailleurs une protection de sécurité de base contre certaines classes d'attaques. La zone des **données statiques** héberge les variables dont la durée de vie couvre toute l'exécution du programme, qu'il s'agisse de variables globales ou de variables locales déclarées `static` (chapitre 4). Le **tas** est une zone de mémoire gérée explicitement par le programmeur via les fonctions d'allocation dynamique : sa taille n'est pas fixée à la compilation et peut croître ou diminuer pendant l'exécution, selon les besoins. La **pile**, enfin, croît et diminue automatiquement à chaque appel et retour de fonction, selon une logique dernier entré/premier sorti (LIFO) : c'est là que vivent les variables locales « ordinaires » et les paramètres de chaque fonction en cours d'exécution.

Comprendre précisément dans quelle zone se trouve une donnée donnée est indispensable pour analyser les grandes classes de vulnérabilités mémoire : un **débordement de pile** (*stack overflow*, au sens d'écriture hors limites d'un buffer local, à ne pas confondre avec le débordement par récursivité vu au chapitre 4) écrit au-delà d'un tableau local et corrompt la pile, potentiellement jusqu'à l'adresse de retour de la fonction ; un **use-after-free** ou une **fuite mémoire** concernent spécifiquement le tas et sa gestion manuelle ; un accès à une variable dont la portée est terminée (un **dangling pointer**, voir section 4) concerne typiquement une variable qui vivait sur la pile.

aux variables globales (dont la taille est également fixée à la compilation), l'**allocation dynamique** permet de réserver, pendant l'exécution du programme, un bloc de mémoire dont la taille peut dépendre de données connues seulement à ce moment-là (par exemple une taille saisie par l'utilisateur), et dont la durée de vie est entièrement contrôlée par le programmeur, indépendamment de la structure des blocs de code.

```
#include <stdlib.h>

int *tableau = malloc(10 * sizeof(int));
if (tableau == NULL) {
    // échec d'allocation : toujours vérifier
    exit(EXIT_FAILURE);
}
for (int i = 0; i < 10; i++) tableau[i] = i * i;

free(tableau);
tableau = NULL; // évite un « dangling pointer »
```

Quatre fonctions de la bibliothèque standard, déclarées dans `stdlib.h`, gèrent l'allocation dynamique sur le tas :

- **malloc(taille)** alloue un bloc de `taille` octets sur le tas et renvoie un pointeur vers son premier octet, ou `NULL` si l'allocation échoue (mémoire insuffisante, par exemple). Le contenu du bloc alloué n'est **pas initialisé** : il contient des valeurs indéterminées, exactement comme une variable locale non initialisée (chapitre 2) — l'utiliser sans l'initialiser au préalable expose aux mêmes risques de lecture de données imprévisibles.
- **calloc(n, taille)** alloue un bloc capable de contenir `n` éléments de `taille` octets chacun, et l'initialise explicitement à zéro, contrairement à `malloc`. Ce comportement en fait un choix plus sûr par défaut lorsque l'initialisation à zéro est acceptable, au prix d'un coût légèrement supérieur.
- **realloc(ptr, nouvelle_taille)** redimensionne un bloc précédemment alloué (par `malloc`, `calloc` ou un `realloc` antérieur), en conservant autant que possible son contenu existant. Le bloc peut être déplacé en mémoire si nécessaire : la valeur de retour de `realloc` doit donc systématiquement remplacer l'ancien pointeur — ne jamais réutiliser l'ancien pointeur après un `realloc`, et prendre garde à ne pas l'écraser directement (`ptr = realloc(ptr, ...)` perd l'ancien pointeur si l'appel échoue et renvoie `NULL`, provoquant une fuite mémoire ; il est donc plus sûr d'utiliser une variable temporaire).
- **free(ptr)** libère un bloc précédemment alloué, rendant la mémoire correspondante disponible pour de futures allocations. Deux règles absolues encadrent son usage : ne **jamais** appeler `free` deux fois sur le même pointeur (**double free**), et ne **jamais** utiliser un pointeur après l'avoir libéré (**use-after-free**) — ces deux erreurs sont détaillées à la section suivante.

Vérifier systématiquement la valeur de retour de `malloc` (ou `calloc/realloc`) avant de l'utiliser n'est pas une précaution optionnelle : sur un

4. Classes de vulnérabilités mémoire liées aux pointeurs

Vulnérabilité	Cause
Déréférencement de pointeur nul	oubli de vérifier le retour de <code>malloc</code> , ou déréférencement d'une variable pointeur non initialisée
Fuite mémoire (<i>memory leak</i>)	<code>malloc</code> (ou équivalent) sans <code>free</code> correspondant : la mémoire reste réservée alors qu'elle n'est plus utilisée
Use-after-free	utilisation (lecture, écriture, ou nouveau <code>free</code>) d'un pointeur après que la mémoire qu'il désignait a été libérée
Double free	<code>free</code> appelé deux fois sur le même pointeur, corrompant les structures internes de gestion du tas
Dangling pointer	pointeur qui continue de désigner l'adresse d'une variable locale dont la portée (et donc la durée de vie sur la pile) est déjà terminée
Buffer overflow	écriture au-delà de la mémoire réellement allouée à un tableau ou un bloc dynamique

Ces six classes de vulnérabilités ne sont pas des curiosités académiques : elles figurent, année après année, parmi les catégories de failles logicielles les plus fréquemment exploitées dans des logiciels réels écrits en C ou C++, précisément parce que ce sont des langages qui laissent au programmeur l'entière responsabilité de la gestion de la mémoire, sans filet de sécurité automatique. Une **fuite mémoire**, prise isolément, dégrade progressivement les performances d'un programme de longue durée (un serveur, par exemple) jusqu'à épuiser la mémoire disponible ; un **use-after-free** ou un **double free**, en revanche, corrompent des structures internes de gestion du tas d'une manière qui peut, dans certaines conditions, être détournée par un attaquant pour exécuter du code arbitraire — c'est l'une des techniques d'exploitation les plus étudiées en analyse de vulnérabilités. Un **dangling pointer** typique survient lorsqu'une fonction renvoie l'adresse d'une de ses variables locales : cette variable n'existe plus une fois la fonction terminée, mais le pointeur renvoyé continue pourtant de pointer vers cette adresse désormais invalide.

Cet ensemble de classes de vulnérabilités constitue le socle indispensable du cours *Sécurité des applications* (où elles seront exploitées de façon contrôlée en environnement pédagogique) et sont directement liées aux outils d'audit statique et dynamique (analyseurs de code, détecteurs de fuites mémoire tels qu'AddressSanitizer ou Valgrind) présentés dans le cours *Audit organisation et technique*.

5. Arithmétique des pointeurs

```
int tab[5] = {10, 20, 30, 40, 50};
int *p = tab;      // p pointe sur tab[0]
printf("%d\n", *(p + 2)); // 30, équivalent à tab[2]
p++;               // p pointe maintenant sur tab[1]
```

Un pointeur peut être combiné avec des opérateurs arithmétiques (+, -, ++, --) pour se déplacer d'une adresse à une autre — c'est précisément ce mécanisme qui, comme vu au chapitre 5, rend équivalentes les écritures `tab[i]` et `*(tab + i)`. Un point essentiel, et fréquemment source de confusion, est que cette arithmétique ne s'effectue **pas** en octets bruts, mais en **unités de la taille du type pointé** : sur un pointeur `int *` (généralement 4 octets par élément), l'expression `p + 1` avance l'adresse contenue dans `p` de `sizeof(int)` octets, et non d'un seul octet, de façon à toujours pointer sur l'élément suivant du tableau, quel que soit le type manipulé. Cette conversion automatique est effectuée par le compilateur en fonction du type déclaré du pointeur ; c'est aussi pourquoi il n'existe, en C standard, aucune arithmétique valide sur un pointeur `void *` (dont le type pointé, et donc la taille, est indéterminé) sans conversion préalable explicite vers un type concret.

6. Pointeurs de fonction (aperçu)

```
int addition(int a, int b) { return a + b; }
int (*operation)(int, int) = addition;
printf("%d\n", operation(2, 3)); // 5
```

Au-delà de pointer vers des données, un pointeur peut également désigner l'**adresse d'une fonction** : le nom d'une fonction, dans une expression, se comporte comme un pointeur vers son code exécutable, exactement comme le nom d'un tableau se comporte comme un pointeur vers son premier élément. La déclaration `int (*operation)(int, int)` définit une variable `operation` capable de stocker l'adresse de n'importe quelle fonction prenant deux `int` en paramètres et renvoyant un `int` — ici, on lui affecte l'adresse de `addition`, ce qui permet ensuite d'appeler cette fonction indirectement, via `operation(2, 3)`, exactement comme un appel direct.

Ce mécanisme est largement utilisé pour construire des **callbacks** (passer une fonction en paramètre d'une autre, pour qu'elle soit appelée à un moment précis — par exemple une fonction de comparaison passée à un algorithme de tri générique) et des **tables de dispatch** (un tableau de pointeurs de fonctions, indexé pour sélectionner dynamiquement le traitement à exécuter selon un code ou un type de message). C'est un mécanisme puissant et légitime, mais qui, mal protégé, constitue également la cible de certaines techniques d'attaque avancées visant à **détourner le flux d'exécution** d'un programme : si un attaquant parvient, via une autre vulnérabilité mémoire (par exemple un débordement de tampon adjacent), à corrompre la valeur stockée dans un pointeur de fonction avant qu'il ne soit appelé, il peut rediriger l'exécution du programme vers du code de son choix. Ce point sera repris dans le cadre de l'étude des techniques d'exploitation en *Sécurité des applications*.

À retenir

- Un pointeur stocke une adresse mémoire ; `&` obtient l'adresse d'une variable, `*` déréférence un pointeur pour lire ou modifier la valeur pointée.
- La mémoire d'un programme s'organise en quatre zones (code, données statiques, tas, pile), chacune avec sa propre durée de vie ; comprendre cette organisation est indispensable pour analyser les vulnérabilités mémoire.
- Toujours vérifier le retour de `malloc/calloc/realloc` avant utilisation, et toujours réaffecter le résultat de `realloc` avec prudence pour ne pas perdre le pointeur d'origine en cas d'échec.
- Toute allocation dynamique doit avoir exactement une libération correspondante ; mettre un pointeur à `NULL` après `free` limite les risques de use-after-free et de double free.
- L'arithmétique de pointeurs s'exprime en unités de la taille du type pointé, jamais en octets bruts.

Chapitre 7 — Structures (struct) et fichiers

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définir une structure

```
struct Utilisateur {
    char nom[50];
    int age;
    float solde;
};

struct Utilisateur u1 = {"Zakaria", 30, 1500.0f};
printf("%s a %d ans\n", u1.nom, u1.age);
```

Une **structure** (`struct`) regroupe plusieurs données de types potentiellement différents — ici une chaîne de caractères, un entier et un flottant — sous un seul type composite nommé, dont chaque instance (chaque variable de ce type) réserve en mémoire un bloc contenant l'ensemble de ces champs, généralement dans l'ordre de leur déclaration. C'est une brique de base essentielle pour modéliser des objets métier cohérents : un utilisateur, un paquet réseau, un enregistrement de journal (*log*), un en-tête de fichier, ou toute autre entité dont la description naturelle combine plusieurs informations liées entre elles. L'accès à un champ d'une structure se fait avec l'opérateur point (`.`) suivi du nom du champ, comme dans `u1.nom` ou `u1.age`.

Il est important de noter que le compilateur peut insérer, entre les champs d'une structure, des octets de remplissage (*padding*) pour respecter les contraintes d'alignement mémoire du processeur cible (par exemple, aligner un `int` sur une adresse multiple de 4). La taille totale d'une structure, mesurée avec `sizeof`, n'est donc pas nécessairement égale à la somme exacte des tailles de ses champs pris individuellement — un point qui aura son importance à la section 6, lors de la sérialisation binaire.

2. Structures et pointeurs

```
struct Utilisateur *pu = &u1;
printf("%s\n", pu->nom);    // équivalent à (*pu).nom
```

Lorsqu'on manipule une structure via un pointeur — ce qui est extrêmement fréquent en C, notamment pour éviter de copier une structure volumineuse à chaque passage en paramètre de fonction (rappel du chapitre 4 : le passage par valeur copie intégralement l'argument) — accéder à un champ requiert de d'abord déréférencer le pointeur avant d'appliquer l'opérateur point : `(*pu).nom`. Cette écriture, correcte mais un peu lourde et nécessitant des parenthèses (sans elles, `*pu.nom` serait interprété comme `*(pu.nom)`, ce qui n'a pas de sens ici), est si courante que le C fournit un opérateur dédié, la **flèche** `->`, qui combine en une seule syntaxe le déréférencement et l'accès au champ : `pu->nom` est strictement équivalent à `(*pu).nom`, mais bien plus lisible.

L'opérateur `->` est, dans la pratique, l'un des opérateurs les plus fréquemment rencontrés dans du code C réel : il est omniprésent dans la manipulation des **listes chaînées** et des **arbres** (voir section 4), ainsi que dans le code des systèmes d'exploitation et des piles réseau, où l'on manipule constamment des structures via des pointeurs — par exemple pour accéder aux champs d'un en-tête de paquet réseau à partir d'un pointeur vers un buffer reçu, un motif que l'on retrouvera dans les cours abordant l'analyse de protocoles.

3. `typedef` et alias de type

```
typedef struct {  
    int x;  
    int y;  
} Point;  
  
Point p = {3, 4};
```

Le mot-clé `typedef` permet de créer un **alias** pour un type existant, souvent complexe ou verbeux à écrire, afin de simplifier son utilisation dans le reste du code. Dans l'exemple ci-dessus, `typedef struct { ... } Point;` définit une structure **anonyme** (sans nom propre après le mot-clé `struct`) et lui donne immédiatement l'alias `Point`, qui peut ensuite être utilisé directement comme un nom de type, sans avoir besoin d'écrire `struct` à chaque déclaration (`Point p = {3, 4};` au lieu du plus verbeux `struct Point p = {3, 4};` que nécessiterait une structure nommée classique). Cette convention, très répandue en C, améliore la lisibilité du code et rapproche visuellement l'usage d'une structure de celui d'un type de base comme `int` ou `double`. `typedef` peut d'ailleurs être utilisé pour créer un alias vers n'importe quel type, pas seulement des structures — par exemple `typedef unsigned char octet;` pour donner un nom métier explicite à un type existant.

4. Structures de données dynamiques : liste chaînée (introduction)

```
typedef struct Noeud {
    int valeur;
    struct Noeud *suivant;
} Noeud;

Noeud* creer_noeud(int v) {
    Noeud *n = malloc(sizeof(Noeud));
    if (n == NULL) return NULL;
    n->valeur = v;
    n->suivant = NULL;
    return n;
}
```

Une **liste chaînée** est une structure de données dans laquelle chaque élément, appelé **nœud**, contient une valeur ainsi qu'un pointeur vers le nœud suivant dans la séquence (ou `NULL` s'il s'agit du dernier nœud). Contrairement à un tableau, dont la taille et l'organisation en mémoire sont fixées à l'allocation, une liste chaînée peut grandir ou rétrécir dynamiquement, un nœud à la fois, sans jamais avoir besoin de déplacer les nœuds existants — au prix de ne plus pouvoir accéder directement à un élément par son indice (il faut parcourir la liste nœud par nœud depuis le début).

Cette définition mérite une observation syntaxique importante : à l'intérieur de la structure `Noeud`, le champ `suivant` est déclaré comme `struct Noeud *suivant`, en utilisant explicitement `struct Noeud` (et non simplement `Noeud`, l'alias, qui n'est pas encore complètement défini à ce point de la déclaration) — c'est une **référence à soi-même**, rendue possible uniquement parce qu'il s'agit d'un *pointeur* vers un `Noeud` et non d'un `Noeud` directement inclus par valeur (ce qui, lui, serait impossible : une structure ne peut pas se contenir elle-même, sa taille serait indéfinie).

La fonction `creer_noeud` illustre la combinaison de trois notions vues précédemment dans ce module, qui convergent ici naturellement : l'**allocation dynamique** (`malloc(sizeof(Noeud))`), avec la vérification systématique de son résultat — chapitre 6), les **pointeurs** (le nœud retourné, et le champ `suivant` qui permettra de chaîner les nœuds entre eux) et les **structures** (le regroupement de `valeur` et `suivant` dans un seul type cohérent). C'est précisément cette combinaison qui rend possible la construction de structures de données dynamiques plus élaborées — listes doublement chaînées, piles, files, arbres binaires — qui seront explorées en travaux pratiques.

```

FILE *f = fopen("donnees.txt", "w");
if (f == NULL) {
    perror("Erreur d'ouverture");
    return 1;
}
fprintf(f, "Bonjour %s\n", "Zakaria");
fclose(f);

FILE *lecture = fopen("donnees.txt", "r");
char ligne[100];
while (fgets(ligne, sizeof(ligne), lecture) != NULL) {
    printf("%s", ligne);
}
fclose(lecture);

```

La bibliothèque standard `stdio.h` représente un fichier ouvert par un pointeur opaque de type `FILE *`, obtenu par `fopen(chemin, mode)`. Le second argument, le **mode d'ouverture**, détermine à la fois l'opération autorisée et l'effet sur le fichier existant : `"r"` ouvre en lecture seule (échoue si le fichier n'existe pas), `"w"` ouvre en écriture en **écrasant** tout contenu préexistant (ou en créant le fichier s'il n'existe pas), `"a"` ouvre en mode ajout (*append*), écrivant à la suite du contenu existant sans l'effacer, et les variantes `"rb"/"wb"` ouvrent respectivement en lecture et en écriture en **mode binaire**, sans les traitements spécifiques que certains systèmes appliquent aux fichiers texte (comme la conversion des fins de ligne).

`fopen` peut échouer pour de nombreuses raisons — fichier inexistant, permissions insuffisantes, disque plein, chemin invalide — et renvoie alors `NULL` plutôt qu'un pointeur valide. Comme pour `malloc`, cette valeur de retour **doit toujours être vérifiée** avant tout usage : tenter d'écrire ou de lire à travers un pointeur `FILE *` nul provoque un comportement indéfini. La fonction `perror`, utilisée ici, affiche sur la sortie d'erreur standard un message décrivant la dernière erreur système survenue, ce qui facilite grandement le diagnostic.

Pour la lecture, `fgets(ligne, sizeof(ligne), lecture)` lit **au maximum** `sizeof(ligne) - 1` caractères depuis le fichier (en s'arrêtant plus tôt si elle rencontre une fin de ligne), les stocke dans `ligne`, et ajoute automatiquement un `'\0'` final — contrairement à `gets` (fonction historique aujourd'hui supprimée du standard C11 précisément pour cette raison), `fgets` ne peut jamais déborder du buffer fourni, car elle connaît explicitement sa taille grâce au second paramètre. Elle renvoie `NULL` lorsqu'il n'y a plus rien à lire (fin de fichier atteinte) ou en cas d'erreur, ce qui permet d'utiliser directement sa valeur de retour comme condition de boucle, comme dans l'exemple.

Point de vigilance sécurité : trois réflexes systématiques s'imposent pour toute manipulation de fichier en C — toujours vérifier la valeur de retour de `fopen` avant de l'utiliser ; toujours borner explicitement la taille lue avec des fonctions comme `fgets(buffer, taille, f)` plutôt que d'utiliser

6. Lecture/écriture binaire

```
struct Utilisateur u;  
fwrite(&u, sizeof(struct Utilisateur), 1, f);  
fread(&u, sizeof(struct Utilisateur), 1, f);
```

Alors que `fprintf/fgets` manipulent du texte lisible, `fwrite` et `fread` permettent d'écrire et de lire directement la **représentation binaire brute** en mémoire d'une variable — ici, l'intégralité des octets qui composent la structure `u`, tels qu'ils sont réellement organisés en mémoire (y compris, le cas échéant, les octets de padding évoqués à la section 1). Les paramètres de `fwrite` (identiques pour `fread`) suivent le même schéma : l'adresse des données (`&u`), la taille en octets d'un élément (`sizeof(struct Utilisateur)`), le nombre d'éléments à écrire ou lire (`1` ici, mais on peut écrire/lire un tableau entier de structures en une seule fois), et le flux de fichier concerné. Cette approche est nettement plus compacte et rapide que de formater chaque champ individuellement en texte, ce qui en fait le choix naturel pour **sérialiser** — c'est-à-dire convertir en une suite d'octets stockable ou transmissible — des structures de données destinées à être relues plus tard, potentiellement par le même programme.

Cette efficacité a toutefois une contrepartie importante : la représentation binaire brute d'une structure dépend directement de l'architecture et du compilateur qui l'ont produite — la taille exacte des types (chapitre 2), la présence et la disposition du padding entre les champs (section 1), et l'**endianness** (l'ordre dans lequel les octets d'une valeur multi-octets, comme un `int`, sont rangés en mémoire — poids fort en premier ou poids faible en premier selon le processeur) peuvent tous varier d'un système à l'autre. Un fichier binaire produit par `fwrite` sur une machine n'est donc pas garanti d'être relisible correctement par `fread` sur une machine d'architecture différente, sauf à normaliser explicitement ces aspects avant l'écriture. Ce sujet, en apparence spécifique au C, sera repris sous un angle différent dans le chapitre *Cryptographie* (où l'encodage précis des données avant chiffrement doit être maîtrisé bit par bit) et dans le cours *Technologie Blockchain* (où la sérialisation déterministe et portable des transactions est une exigence de conception centrale).

À retenir

- `struct` modélise des données composites hétérogènes sous un seul type ; combinée à des pointeurs, elle permet de construire des structures de données dynamiques (listes chaînées, arbres) via l'opérateur `->`.
- `typedef` simplifie l'utilisation des types complexes, notamment les structures, en leur donnant un alias direct sans répéter le mot-clé `struct`.
- La gestion de fichiers en C impose une vérification systématique des erreurs (`fopen`, `fread`, `fwrite`), une lecture toujours bornée en taille, une fermeture explicite (`fclose`), et la validation de tout chemin construit à partir d'une entrée utilisateur.
- La sérialisation binaire (`fwrite/fread`) est efficace mais sensible aux différences d'alignement mémoire et d'endianness entre systèmes.
- Ce chapitre clôt le socle du langage : les travaux pratiques suivants consolident l'ensemble de ces notions — variables, structures de contrôle, fonctions, tableaux, pointeurs, structures et fichiers — sur des cas pratiques complets.